Deep Q-Networks (DQN) 🧠

The core idea of DQN is to train a **Neural Network** to approximate the State-Action Value Function, $Q(s, a)$. Once we have this network, we can input any state s and know exactly which action a yields the highest return.

The Neural Network Architecture

We design a network to predict the Q-value.

- **Input (X):** A vector of **12 numbers**.
  - **State (s):** 8 numbers (position, velocity, angle, leg sensors).
  - **Action (a):** 4 numbers (One-Hot Encoded vector).
    - *Nothing:* `[1, 0, 0, 0]`
    - *Left:* `[0, 1, 0, 0]`
    - *Main:* `[0, 0, 1, 0]`
    - *Right:* `[0, 0, 0, 1]`
- **Hidden Layers:**
  - Layer 1: 64 units.
  - Layer 2: 64 units.
- **Output (Y):** A single number representing $Q(s, a)$.

Creating the Training Set

Since this is Reinforcement Learning, we don't start with a labeled dataset of "correct answers." We have to create our own data by experimenting.

1. **Exploration (Replay Buffer):**
   - The lander flies around (initially randomly).
   - We record every step as a **tuple**: (s, a, R, s').
     - *Example:* (State A, Fire Main Engine, Reward -0.3, State B).
   - We store the most recent **10,000 tuples** in a memory bank called the **Replay Buffer**.
2. **Generating Targets (Y):**
   - To train the network, we need an input X and a target output Y.
   - **Input X:** The state and action from the tuple (s, a).
   - **Target Y:** Calculated using the **Bellman Equation**:
     $Y = R(s) + \gamma \max_{a'} Q(s', a')$

- *Crucial Note:* Since we don't know the true Q yet, we use our **current neural network** to estimate the $Q(s', a')$ part. It's a guess, but it gets better over time.

The Full DQN Algorithm

1. **Initialize:** Create the Neural Network with random weights (it knows nothing).
2. **Loop (Repeat forever):**
   - **Act:** Play the game. Choose actions (sometimes random, sometimes using the current network).
   - **Store:** Save the experience (s, a, R, s') into the **Replay Buffer**.
   - **Train:**
     - Grab a batch of recent experiences from the buffer.
     - Create training examples: $X = (s, a), Y = R + \gamma \max Q(s', a')$.
     - Train the network to minimize the error between its prediction and Y (using Mean Squared Error).
   - **Update:** The network is now slightly smarter. Use this smarter network for the next loop.

Why This Works

- At first, the network's guesses are garbage.
- However, the **Immediate Reward (R)** is a fact (ground truth).
- By constantly training the network to match "R + future guess," the truth of R slowly propagates backward through the states.
- Over thousands of updates, the network eventually learns to accurately predict the Q-value for every state.